Week-10 - Advanced CPU Scheduling Assignments

Name: Aditya Kumar

Registration no: AP24110010333

BTech CSE - C

Assignment 1: SRTF with Context Switch Overhead

Question:

Write a program that simulates the Shortest Remaining Time First (SRTF) algorithm when every context switch takes a fixed time C, during which no process executes. A newly arrived process Pnew preempts the running process Pcurr only if remaining(Pcurr) > remaining(Pnew) + C. Print the Gantt chart including the context switch periods, CT, TAT and WT of each process, the number of context switches, the total time spent on switches and the CPU efficiency (total busy time / total timeline length × 100).

Input format:

```
n
pid arrival_time burst_time (n lines)
C
```

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAXP 20
#define MAXT 5000

int n, C;
char pid[MAXP][8];
int at[MAXP], bt[MAXP], rem[MAXP], ct[MAXP];
char lab[MAXT][8];   // what the CPU did in each time unit
int brk[MAXT];       // 1 if a new Gantt segment starts at this unit
int running = -1;    // process on the CPU (-1 = none)

// ready = arrived, unfinished and not on the CPU
int pick_shortest(int t) {
    int best = -1;
    for (int i = 0; i < n; i++) {
        if (at[i] > t || rem[i] == 0 || i == running) continue;
        if (best == -1 || rem[i] < rem[best] ||
            (rem[i] == rem[best] && at[i] < at[best]))
            best = i;
    }
    return best;
}

void mark(int t, const char *name, int newSeg) {
    strcpy(lab[t], name);
    brk[t] = newSeg || t == 0 || strcmp(lab[t - 1], name) != 0;
}

int main(void) {
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
        scanf("%7s %d %d", pid[i], &at[i], &bt[i]);
        rem[i] = bt[i];
    }
    scanf("%d", &C);

    printf("Process  AT  BT\n");
    for (int i = 0; i < n; i++) printf("%-7s %3d %3d\n", pid[i], at[i], bt[i]);
    printf("Context switch overhead C = %d\n\n", C);

    int t = 0, done = 0, csLeft = 0, csCount = 0, csTime = 0, busy = 0;
    while (done < n) {
        if (csLeft > 0) {                          // CPU is switching
            mark(t, "CS", csLeft == C);
            csLeft--; csTime++; t++;
            if (csLeft == 0) running = pick_shortest(t);   // dispatch when the switch ends
            continue;
        }
        int best = pick_shortest(t);
        if (running == -1) {                       // CPU free
            if (best == -1) {                      // nothing to run: idle
                mark(t, "IDLE", 0);
                t++;
                continue;
            }
            csLeft = C; csCount++;                 // idle/finish -> CS
            continue;
        }
        // preemption test: remaining(curr) > remaining(new) + C
        if (best != -1 && rem[running] > rem[best] + C) {
            running = -1; csLeft = C; csCount++;   // preempted -> CS
            continue;
        }
        mark(t, pid[running], 0);                  // run for one unit
        rem[running]--; busy++; t++;
        if (rem[running] == 0) { ct[running] = t; done++; running = -1; }
    }

    printf("Gantt Chart:\n");
    for (int u = 0; u < t; u++)
        if (brk[u]) printf("%d-%s-", u, lab[u]);
    printf("%d\n\n", t);

    double sumTat = 0, sumWt = 0;
    printf("Process  AT  BT   CT  TAT   WT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        sumTat += tat; sumWt += wt;
        printf("%-7s %3d %3d %4d %4d %4d\n", pid[i], at[i], bt[i], ct[i], tat, wt);
    }
    printf("Average TAT = %.2f, Average WT = %.2f\n\n", sumTat / n, sumWt / n);
    printf("Total context switches = %d\n", csCount);
    printf("Total time on switches = %d\n", csTime);
    printf("Total busy time = %d\n", busy);
    printf("Total timeline length = %d\n", t);
    printf("CPU efficiency = %d / %d x 100 = %.2f%%\n", busy, t, 100.0 * busy / t);
    return 0;
}
```

OUTPUT:

For case-1 (sample input): P1(0,10), P2(4,6), P3(5,2), C = 2

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in1_a.txt
3
P1 0 10
P2 4 6
P3 5 2
2
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./srtf_cs < in1_a.txt
Process  AT  BT
P1        0  10
P2        4   6
P3        5   2
Context switch overhead C = 2

Gantt Chart:
0-CS-2-P1-5-CS-7-P3-9-CS-11-P2-17-CS-19-P1-26

Process  AT  BT   CT  TAT   WT
P1        0  10   26   26   16
P2        4   6   17   13    7
P3        5   2    9    4    2
Average TAT = 14.33, Average WT = 8.33

Total context switches = 4
Total time on switches = 8
Total busy time = 18
Total timeline length = 26
CPU efficiency = 18 / 26 x 100 = 69.23%
```

For case-2: four processes, C = 1

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in1_b.txt
4
P1 0 8
P2 1 4
P3 2 9
P4 3 5
1
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./srtf_cs < in1_b.txt
Process  AT  BT
P1        0   8
P2        1   4
P3        2   9
P4        3   5
Context switch overhead C = 1

Gantt Chart:
0-CS-1-P2-5-CS-6-P4-11-CS-12-P1-20-CS-21-P3-30

Process  AT  BT   CT  TAT   WT
P1        0   8   20   20   12
P2        1   4    5    4    0
P3        2   9   30   28   19
P4        3   5   11    8    3
Average TAT = 15.00, Average WT = 8.50

Total context switches = 4
Total time on switches = 4
```

```
Total busy time = 26
Total timeline length = 30
CPU efficiency = 26 / 30 x 100 = 86.67%
```

For case-3: same processes as case-1 but C = 5

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in1_c.txt
3
P1 0 10
P2 4 6
P3 5 2
5
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./srtf_cs < in1_c.txt
Process  AT  BT
P1        0  10
P2        4   6
P3        5   2
Context switch overhead C = 5

Gantt Chart:
0-CS-5-P3-7-CS-12-P2-18-CS-23-P1-33

Process  AT  BT   CT  TAT   WT
P1        0  10   33   33   23
P2        4   6   18   14    8
P3        5   2    7    2    0
Average TAT = 16.33, Average WT = 10.33

Total context switches = 3
Total time on switches = 15
Total busy time = 18
Total timeline length = 33
CPU efficiency = 18 / 33 x 100 = 54.55%
```

REPORT:

- When a switch happens: a switch of C units is added (1) when the CPU is idle and a process is picked, which includes time 0, (2) when a running process finishes and another one is picked, and (3) when a running process is preempted. The process that actually runs is picked when the switch ends, so a process that arrives during the switch is also considered.
- Preemption test: at every time unit the shortest ready process is compared with the running one, and the running process is preempted only if remaining(Pcurr) > remaining(Pnew) + C.
- Case-1: P1 starts after the first switch (0-2). At t = 4, P2 (6) arrives while P1 has 8 left: 8 > 6 + 2 is false, so there is no preemption. At t = 5, P3 (2) arrives while P1 has 7 left: 7 > 2 + 2 is true, so P1 is preempted. The order is P1, P3, P2, P1 with 4 context switches taking 8 units, and the efficiency is 18 / 26 = 69.23%.
- Effect of C: case-3 uses the same processes as case-1 with C = 5. The first switch ends at t = 5, by which time P2 and P3 have arrived, so P3 runs first and P1 runs last. There are only 3 switches, but they take 15 units, so the efficiency falls from 69.23% to 54.55%. A larger C makes the CPU spend more of its time switching.

Assignment 2: Round Robin with Dynamic Time Quantum

Question:

Write a program that implements Round Robin with a time quantum that changes from cycle to cycle. In the first cycle TQ is the median of the burst times of the processes in the initial ready queue. After every cycle,

TQ(next) = ceil(average burst time of the processes in the new ready queue / 2). If the ready queue is empty, the previous TQ is kept for the next arrival. New arrivals go to the back of the queue and affect only the next TQ. Print the Gantt chart, CT, TAT and WT with averages, and a table of cycle number, TQ used and the processes in the ready queue.

Input format:

```
n
pid arrival_time burst_time (n lines)
```

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAXP 20
#define MAXQ 5000
#define MAXS 5000

int n;
char pid[MAXP][8];
int at[MAXP], bt[MAXP], rem[MAXP], ct[MAXP], admitted[MAXP];
int q[MAXQ], head = 0, tail = 0;                    // ready queue
int sgS[MAXS], sgE[MAXS], sgP[MAXS], sgN = 0;       // Gantt segments (-1 = idle)
// cycle table
int cycTq[MAXS], cycN[MAXS], cycMem[MAXS][MAXP], cycRem[MAXS][MAXP];

void seg(int s, int e, int p) {
    if (sgN > 0 && sgP[sgN - 1] == p && sgE[sgN - 1] == s) { sgE[sgN - 1] = e; return; }
    sgS[sgN] = s; sgE[sgN] = e; sgP[sgN] = p; sgN++;
}

// put every process that has arrived by time t at the back of the queue
void admit(int t) {
    for (;;) {
        int b = -1;
        for (int i = 0; i < n; i++)
            if (!admitted[i] && at[i] <= t && (b == -1 || at[i] < at[b])) b = i;
        if (b == -1) return;
        admitted[b] = 1;
        q[tail++] = b;
    }
}

int median_of_queue(void) {
    int a[MAXP], m = tail - head;
    for (int k = 0; k < m; k++) a[k] = rem[q[head + k]];
    for (int i = 0; i < m; i++)                     // sort
        for (int j = i + 1; j < m; j++)
            if (a[j] < a[i]) { int x = a[i]; a[i] = a[j]; a[j] = x; }
    if (m % 2) return a[m / 2];
    return (a[m / 2 - 1] + a[m / 2] + 1) / 2;       // even count: mean of middle two, rounded
up
}

int main(void) {
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%7s %d %d", pid[i], &at[i], &bt[i]);
        rem[i] = bt[i];
```

```
    }
    printf("Process  AT  BT\n");
    for (int i = 0; i < n; i++) printf("%-7s %3d %3d\n", pid[i], at[i], bt[i]);
    printf("\n");

    int t = 0, done = 0, cycles = 0, prevTq = 1, idled = 0;
    while (done < n) {
        admit(t);
        if (head == tail) {                         // queue empty: wait for next arrival
            int nx = -1;
            for (int i = 0; i < n; i++)
                if (!admitted[i] && (nx == -1 || at[i] < nx)) nx = at[i];
            seg(t, nx, -1);
            t = nx; idled = 1;
            continue;
        }
        int N = tail - head, tq;
        if (cycles == 0) tq = median_of_queue();    // first cycle: median burst
        else if (idled) tq = prevTq;                // queue was empty: keep previous TQ
        else {                                      // ceil(average / 2)
            int sum = 0;
            for (int k = head; k < tail; k++) sum += rem[q[k]];
            tq = (sum + 2 * N - 1) / (2 * N);
        }
        if (tq < 1) tq = 1;
        idled = 0;

        cycTq[cycles] = tq; cycN[cycles] = N;
        for (int k = 0; k < N; k++) {
            cycMem[cycles][k] = q[head + k];
            cycRem[cycles][k] = rem[q[head + k]];
        }
        cycles++;

        for (int k = 0; k < N; k++) {               // one pass over the N processes
            int i = q[head++];
            int run = rem[i] < tq ? rem[i] : tq;
            seg(t, t + run, i);
            t += run; rem[i] -= run;
            admit(t);                               // arrivals go behind the queue
            if (rem[i] > 0) q[tail++] = i;          // preempted: back of the queue
            else { ct[i] = t; done++; }
        }
        prevTq = tq;
    }

    printf("Gantt Chart:\n");
    for (int s = 0; s < sgN; s++)
        printf("%d-%s-", sgS[s], sgP[s] < 0 ? "IDLE" : pid[sgP[s]]);
    printf("%d\n\n", t);

    printf("Cycle | TQ | Processes in RQ      | Remaining times\n");
    for (int c = 0; c < cycles; c++) {
        char names[100] = "[", rems[100] = "[";
        for (int k = 0; k < cycN[c]; k++) {
            char tmp[16];
            strcat(names, pid[cycMem[c][k]]);
            sprintf(tmp, "%d", cycRem[c][k]); strcat(rems, tmp);
            if (k < cycN[c] - 1) { strcat(names, ", "); strcat(rems, ", "); }
        }
        strcat(names, "]"); strcat(rems, "]");
        printf("%5d | %2d | %-20s | %s\n", c + 1, cycTq[c], names, rems);
```

```
    }

    double sumTat = 0, sumWt = 0;
    printf("\nProcess  AT  BT   CT  TAT   WT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        sumTat += tat; sumWt += wt;
        printf("%-7s %3d %3d %4d %4d %4d\n", pid[i], at[i], bt[i], ct[i], tat, wt);
    }
    printf("Average TAT = %.2f, Average WT = %.2f\n", sumTat / n, sumWt / n);
    return 0;
}
```

OUTPUT:

For case-1 (sample input): P1(0,12), P2(1,8), P3(3,18), P4(5,2)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in2_a.txt
4
P1 0 12
P2 1 8
P3 3 18
P4 5 2
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rr_dynamic < in2_a.txt
Process  AT  BT
P1        0  12
P2        1   8
P3        3  18
P4        5   2

Gantt Chart:
0-P1-12-P2-17-P3-22-P4-24-P2-27-P3-40

Cycle | TQ | Processes in RQ      | Remaining times
    1 | 12 | [P1]                 | [12]
    2 |  5 | [P2, P3, P4]         | [8, 18, 2]
    3 |  4 | [P2, P3]             | [3, 13]
    4 |  5 | [P3]                 | [9]
    5 |  2 | [P3]                 | [4]
    6 |  1 | [P3]                 | [2]
    7 |  1 | [P3]                 | [1]

Process  AT  BT   CT  TAT   WT
P1        0  12   12   12    0
P2        1   8   27   26   18
P3        3  18   40   37   19
P4        5   2   24   19   17
Average TAT = 23.50, Average WT = 13.50
```

For case-2: all four processes arrive at time 0

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in2_b.txt
4
P1 0 6
P2 0 3
P3 0 9
P4 0 4
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rr_dynamic < in2_b.txt
Process  AT  BT
P1        0   6
P2        0   3
```

```
P3        0   9
P4        0   4

Gantt Chart:
0-P1-5-P2-8-P3-13-P4-17-P1-18-P3-22

Cycle | TQ | Processes in RQ      | Remaining times
    1 |  5 | [P1, P2, P3, P4]     | [6, 3, 9, 4]
    2 |  2 | [P1, P3]             | [1, 4]
    3 |  1 | [P3]                 | [2]
    4 |  1 | [P3]                 | [1]

Process  AT  BT   CT  TAT   WT
P1        0   6   18   18   12
P2        0   3    8    8    5
P3        0   9   22   22   13
P4        0   4   17   17   13
Average TAT = 16.25, Average WT = 10.75
```

For case-3: the CPU is idle for a while between arrivals

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in2_c.txt
3
P1 0 4
P2 10 6
P3 11 3
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rr_dynamic < in2_c.txt
Process  AT  BT
P1        0   4
P2       10   6
P3       11   3

Gantt Chart:
0-P1-4-IDLE-10-P2-14-P3-16-P2-18-P3-19

Cycle | TQ | Processes in RQ      | Remaining times
    1 |  4 | [P1]                 | [4]
    2 |  4 | [P2]                 | [6]
    3 |  2 | [P3, P2]             | [3, 2]
    4 |  1 | [P3]                 | [1]

Process  AT  BT   CT  TAT   WT
P1        0   4    4    4    0
P2       10   6   18    8    2
P3       11   3   19    8    5
Average TAT = 6.67, Average WT = 2.33
```

REPORT:

- Cycle: one cycle is one pass over the processes that are in the ready queue when the cycle starts. A process that is not finished goes to the back of the queue. Processes that arrive during the cycle are added behind it and join the next cycle.
- Choices where the question is open: the initial ready queue is the set of processes that have arrived when the CPU first starts. The average used for TQ(next) is taken over the remaining burst times of the queue. For an even number of processes the median is the mean of the two middle values, rounded up.

- Case-1: only P1 has arrived at t = 0, so the median is 12 and P1 finishes in cycle 1. In cycle 2 the queue holds P2, P3 and P4 with remaining times 8, 18 and 2, so the average is 9.33, half of it is 4.67 and TQ = 5. Later cycles use 4, 5, 2, 1 and 1 as the remaining work shrinks. Averages: TAT = 23.50 and WT = 13.50.
- Case-2: the bursts are 6, 3, 9 and 4, so the median is (4 + 6) / 2 = 5 and TQ = 5 in the first cycle; cycle 2 has P1 and P3 left, whose average is 2.5 and TQ = 2.
- Case-3: the CPU is idle from t = 4 to t = 10. When P2 arrives the queue was empty, so cycle 2 reuses the previous TQ of 4.
- Observation: when one process is left, its TQ keeps halving (5, 2, 1, 1 in case-1), which adds many short cycles. This is a drawback of this rule.

Assignment 3: HRRN with Priority and Anti-Starvation

Question:

Write a non-preemptive scheduler that uses Highest Response Ratio Next with a priority weight. The Adjusted Response Ratio is ARR = (Waiting Time + Burst Time) / (Burst Time × Priority Weight), where Priority Weight = 1 + (PR - 1) × 0.2 and a lower PR means a higher priority. If a process in the ready queue has waited more than the starvation threshold (default 2 × the largest burst time), priority is ignored and the process with the highest standard response ratio (W + B) / B is chosen. Print the order of execution, CT, TAT and WT with averages, details of every scheduling decision and the processes that triggered anti-starvation.

Input format:

```
n
pid arrival_time burst_time priority (n lines)
threshold (0 = use 2 x largest burst time)
```

CODE:

```
#include <stdio.h>

#define MAXP 20

int n;
char pid[MAXP][8];
int at[MAXP], bt[MAXP], pr[MAXP], ct[MAXP], fin[MAXP], trig[MAXP];

int main(void) {
    int thr, order[MAXP], oc = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%7s %d %d %d", pid[i], &at[i], &bt[i], &pr[i]);
    scanf("%d", &thr);                      // 0 = use 2 x max(BT)

    int maxbt = 0;
    for (int i = 0; i < n; i++) if (bt[i] > maxbt) maxbt = bt[i];
    if (thr <= 0) thr = 2 * maxbt;

    printf("Process  AT  BT  PR\n");
    for (int i = 0; i < n; i++) printf("%-7s %3d %3d %3d\n", pid[i], at[i], bt[i], pr[i]);
    printf("Starvation threshold = %d\n\n", thr);

    int t = 0, done = 0, decision = 0;
    while (done < n) {
        int el[MAXP], ne = 0, starved = 0;
        for (int i = 0; i < n; i++)
```

```
            if (!fin[i] && at[i] <= t) { el[ne++] = i; if (t - at[i] > thr) starved = 1; }

        if (ne == 0) {                      // CPU idle until next arrival
            int nx = -1;
            for (int i = 0; i < n; i++)
                if (!fin[i] && (nx == -1 || at[i] < nx)) nx = at[i];
            printf("t = %d: no process is ready, CPU idle until t = %d\n\n", t, nx);
            t = nx;
            continue;
        }

        decision++;
        printf("Decision %d at t = %d\n", decision, t);
        printf("  Eligible: ");
        for (int k = 0; k < ne; k++) printf("%s%s", pid[el[k]], k < ne - 1 ? ", " : "\n");

        int best = -1;
        double bestv = -1;
        for (int k = 0; k < ne; k++) {
            int i = el[k];
            int w = t - at[i];
            double rr = (double)(w + bt[i]) / bt[i];
            double weight = 1.0 + (pr[i] - 1) * 0.2;
            double arr = (double)(w + bt[i]) / (bt[i] * weight);
            int isStarving = w > thr;
            if (isStarving) trig[i] = 1;

            if (starved) {
                printf("  %s: W=%d, RR=(%d+%d)/%d = %.3f%s\n", pid[i], w, w, bt[i], bt[i], rr,
                       isStarving ? " <-- waiting > threshold" : "");
                // only the starving processes compete
                if (isStarving && (best == -1 || rr > bestv ||
                    (rr == bestv && at[i] < at[best]))) { best = i; bestv = rr; }
            } else {
                printf("  %s: W=%d, weight=%.1f, ARR=(%d+%d)/(%d*%.1f) = %.3f\n", pid[i], w,
                       weight, w, bt[i], bt[i], weight, arr);
                if (best == -1 || arr > bestv ||
                    (arr == bestv && at[i] < at[best])) { best = i; bestv = arr; }
            }
        }

        const char *why = starved ? "Anti-starvation: Highest RR"
                        : (ne == 1 ? "Only eligible process" : "Highest ARR");
        printf("  Chosen : %s (%s)\n\n", pid[best], why);

        order[oc++] = best;
        t += bt[best];
        ct[best] = t; fin[best] = 1; done++;
    }

    printf("Order of execution: ");
    for (int k = 0; k < oc; k++) printf("%s%s", pid[order[k]], k < oc - 1 ? " -> " : "\n\n");

    double sumTat = 0, sumWt = 0;
    printf("Process  AT  BT  PR   CT  TAT   WT\n");
    for (int i = 0; i < n; i++) {
        int tat = ct[i] - at[i], wt = tat - bt[i];
        sumTat += tat; sumWt += wt;
        printf("%-7s %3d %3d %3d %4d %4d %4d\n", pid[i], at[i], bt[i], pr[i], ct[i], tat, wt);
    }
    printf("Average TAT = %.2f, Average WT = %.2f\n\n", sumTat / n, sumWt / n);
```

```
    printf("Processes that triggered anti-starvation: ");
    int any = 0;
    for (int i = 0; i < n; i++) if (trig[i]) { printf("%s%s", any ? ", " : "", pid[i]); any =
1; }
    printf("%s\n", any ? "" : "None");
    return 0;
}
```

OUTPUT:

For case-1 (sample input): P1(0,10,PR1), P2(1,50,PR4), P3(2,15,PR2), threshold = 100

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in3_a.txt
3
P1 0 10 1
P2 1 50 4
P3 2 15 2
100
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./hrrn_priority < in3_a.txt
Process  AT  BT  PR
P1        0  10   1
P2        1  50   4
P3        2  15   2
Starvation threshold = 100

Decision 1 at t = 0
  Eligible: P1
  P1: W=0, weight=1.0, ARR=(0+10)/(10*1.0) = 1.000
  Chosen : P1 (Only eligible process)

Decision 2 at t = 10
  Eligible: P2, P3
  P2: W=9, weight=1.6, ARR=(9+50)/(50*1.6) = 0.738
  P3: W=8, weight=1.2, ARR=(8+15)/(15*1.2) = 1.278
  Chosen : P3 (Highest ARR)

Decision 3 at t = 25
  Eligible: P2
  P2: W=24, weight=1.6, ARR=(24+50)/(50*1.6) = 0.925
  Chosen : P2 (Only eligible process)

Order of execution: P1 -> P3 -> P2

Process  AT  BT  PR   CT  TAT   WT
P1        0  10   1   10   10    0
P2        1  50   4   75   74   24
P3        2  15   2   25   23    8
Average TAT = 35.67, Average WT = 10.67

Processes that triggered anti-starvation: None
```

For case-2: six processes with threshold = 15, so anti-starvation is triggered

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in3_b.txt
6
P1 0 5 1
P2 1 20 5
P3 2 4 1
P4 3 6 2
```

```
P5 4 3 1
P6 10 2 1
15
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./hrrn_priority < in3_b.txt
Process  AT  BT  PR
P1        0   5   1
P2        1  20   5
P3        2   4   1
P4        3   6   2
P5        4   3   1
P6       10   2   1
Starvation threshold = 15

Decision 1 at t = 0
  Eligible: P1
  P1: W=0, weight=1.0, ARR=(0+5)/(5*1.0) = 1.000
  Chosen : P1 (Only eligible process)

Decision 2 at t = 5
  Eligible: P2, P3, P4, P5
  P2: W=4, weight=1.8, ARR=(4+20)/(20*1.8) = 0.667
  P3: W=3, weight=1.0, ARR=(3+4)/(4*1.0) = 1.750
  P4: W=2, weight=1.2, ARR=(2+6)/(6*1.2) = 1.111
  P5: W=1, weight=1.0, ARR=(1+3)/(3*1.0) = 1.333
  Chosen : P3 (Highest ARR)

Decision 3 at t = 9
  Eligible: P2, P4, P5
  P2: W=8, weight=1.8, ARR=(8+20)/(20*1.8) = 0.778
  P4: W=6, weight=1.2, ARR=(6+6)/(6*1.2) = 1.667
  P5: W=5, weight=1.0, ARR=(5+3)/(3*1.0) = 2.667
  Chosen : P5 (Highest ARR)

Decision 4 at t = 12
  Eligible: P2, P4, P6
  P2: W=11, weight=1.8, ARR=(11+20)/(20*1.8) = 0.861
  P4: W=9, weight=1.2, ARR=(9+6)/(6*1.2) = 2.083
  P6: W=2, weight=1.0, ARR=(2+2)/(2*1.0) = 2.000
  Chosen : P4 (Highest ARR)

Decision 5 at t = 18
  Eligible: P2, P6
  P2: W=17, RR=(17+20)/20 = 1.850 <-- waiting > threshold
  P6: W=8, RR=(8+2)/2 = 5.000
  Chosen : P2 (Anti-starvation: Highest RR)

Decision 6 at t = 38
  Eligible: P6
  P6: W=28, RR=(28+2)/2 = 15.000 <-- waiting > threshold
  Chosen : P6 (Anti-starvation: Highest RR)

Order of execution: P1 -> P3 -> P5 -> P4 -> P2 -> P6

Process  AT  BT  PR   CT  TAT   WT
P1        0   5   1    5    5    0
P2        1  20   5   38   37   17
P3        2   4   1    9    7    3
P4        3   6   2   18   15    9
P5        4   3   1   12    8    5
P6       10   2   1   40   30   28
Average TAT = 17.00, Average WT = 10.33
```

```
Processes that triggered anti-starvation: P2, P6
```

REPORT:

- Decision rule: the scheduler runs whenever the CPU becomes free. It first checks whether any eligible process has waited longer than the threshold. If none has, it picks the highest ARR; otherwise it picks the process with the highest standard response ratio. If no process has arrived yet, the CPU stays idle until the next arrival.
- Choice where the question is open: when the threshold is crossed, the choice is made among the processes that have crossed it, so that a starving process is certain to be served. Ties are broken by earlier arrival.
- Case-1: P1 runs first. At t = 10, P3 has ARR 1.278 and P2 has ARR 0.738, because P2 has a long burst and a low priority, so P3 runs before P2. The order is P1, P3, P2. The threshold of 100 is never crossed, so anti-starvation is not triggered.
- Case-2: with threshold = 15, P2 (PR 5, BT 20) is passed over four times. At t = 18, P2 has waited 17 > 15. Without the override, P6 would run (ARR 5.0 against 1.03 for P2), but anti-starvation selects P2. P6 then waits 28 units and also crosses the threshold. The processes that triggered the mechanism are P2 and P6. The override keeps the low-priority process from waiting indefinitely, at the cost of delaying a high-priority one.

Assignment 4: Rate Monotonic Scheduling (RMS) Analysis and Simulation

Question:

Write a simulator for Rate Monotonic Scheduling. Each periodic task has a computation time C, a period T (the deadline equals T) and an optional phase. First check schedulability with the Liu-Layland utilization bound U ≤ n(2^(1/n) - 1) and with response-time analysis R(k+1) = C + Σ ceil(R(k) / Tj) × Cj over higher-priority tasks. Then simulate one hyperperiod (LCM of the periods), showing which task instance runs at each time, and list every missed deadline.

Input format:

```
n
task_id C T phase (n lines, phase 0 if none)
```

CODE:

```
#include <stdio.h>
#include <math.h>
#include <string.h>

#define MAXT 10
#define MAXJ 3000

int n;
char id[MAXT][8];
int Cc[MAXT], Tp[MAXT], ph[MAXT];

typedef struct { int task, num, rel, rem, dl, fin, missed, remAtMiss; } Job;
Job jobs[MAXJ];
int nj = 0;

long gcd(long a, long b) { return b ? gcd(b, a % b) : a; }
```

```
int main(void) {
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%7s %d %d %d", id[i], &Cc[i], &Tp[i], &ph[i]);

    // priority order: shorter period = higher priority (stable sort)
    for (int i = 1; i < n; i++) {
        char s[8]; int c = Cc[i], p = Tp[i], f = ph[i], j = i - 1;
        strcpy(s, id[i]);
        while (j >= 0 && Tp[j] > p) {
            strcpy(id[j + 1], id[j]); Cc[j + 1] = Cc[j]; Tp[j + 1] = Tp[j]; ph[j + 1] = ph[j];
            j--;
        }
        strcpy(id[j + 1], s); Cc[j + 1] = c; Tp[j + 1] = p; ph[j + 1] = f;
    }

    printf("Task   C    T  Phase (listed in priority order)\n");
    for (int i = 0; i < n; i++) printf("%-5s %2d %4d %6d\n", id[i], Cc[i], Tp[i], ph[i]);

    // ---------- Step 1: schedulability analysis ----------
    double U = 0;
    for (int i = 0; i < n; i++) U += (double)Cc[i] / Tp[i];
    double bound = n * (pow(2.0, 1.0 / n) - 1);

    printf("\n--- Schedulability analysis ---\n");
    printf("Utilization U = ");
    for (int i = 0; i < n; i++) printf("%d/%d%s", Cc[i], Tp[i], i < n - 1 ? " + " : "");
    printf(" = %.4f\n", U);
    printf("Liu-Layland bound n(2^(1/n)-1) = %.4f\n", bound);
    if (U > 1.0) printf("Utilization test: U > 1, definitely UNSCHEDULABLE\n");
    else if (U <= bound) printf("Utilization test: U <= bound, schedulability GUARANTEED\n");
    else printf("Utilization test: bound < U <= 1, NOT GUARANTEED (inconclusive, check
RTA)\n");

    printf("\nResponse-time analysis:\n");
    int R[MAXT], allOk = 1;
    for (int i = 0; i < n; i++) {
        int r = Cc[i];
        printf("  %s: R = %d", id[i], r);
        for (;;) {
            int next = Cc[i];
            for (int j = 0; j < i; j++) next += ((r + Tp[j] - 1) / Tp[j]) * Cc[j];
            printf(" -> %d", next);
            if (next == r) break;
            r = next;
            if (r > Tp[i]) break;           // deadline miss
        }
        R[i] = r;
        int ok = r <= Tp[i];
        if (!ok) allOk = 0;
        printf(" => R = %d, deadline = %d : %s\n", r, Tp[i], ok ? "schedulable" : "MISSES
DEADLINE");
    }
    printf("RTA conclusion: task set is %s\n", allOk ? "SCHEDULABLE" : "NOT schedulable");

    // ---------- Step 2: timeline simulation over one hyperperiod ----------
    long H = Tp[0], maxph = 0;
    for (int i = 1; i < n; i++) H = H / gcd(H, Tp[i]) * Tp[i];
    for (int i = 0; i < n; i++) if (ph[i] > maxph) maxph = ph[i];
    int simLen = (int)(H + maxph);
```

```
    for (int i = 0; i < n; i++) {
        int k = 1;
        for (int r = ph[i]; r < simLen; r += Tp[i], k++) {
            Job *j = &jobs[nj++];
            j->task = i; j->num = k; j->rel = r; j->rem = Cc[i];
            j->dl = r + Tp[i]; j->fin = -1; j->missed = 0;
        }
    }

    static char lab[20000][12];
    static int brk[20000];
    int missOrder[MAXJ], nm = 0;

    for (int t = 0; t < simLen; t++) {
        for (int k = 0; k < nj; k++)        // deadline check at time t
            if (!jobs[k].missed && jobs[k].rem > 0 && jobs[k].dl <= t) {
                jobs[k].missed = 1; jobs[k].remAtMiss = jobs[k].rem; missOrder[nm++] = k;
            }
        int pick = -1;
        for (int k = 0; k < nj; k++)
            if (jobs[k].rel <= t && jobs[k].rem > 0 &&
                (pick == -1 || jobs[k].task < jobs[pick].task)) pick = k;

        if (pick == -1) strcpy(lab[t], "IDLE");
        else sprintf(lab[t], "%s.%d", id[jobs[pick].task], jobs[pick].num);
        brk[t] = t == 0 || strcmp(lab[t], lab[t - 1]) != 0;
        if (pick != -1 && --jobs[pick].rem == 0) jobs[pick].fin = t + 1;
    }
    for (int k = 0; k < nj; k++)            // deadlines falling exactly at the end
        if (!jobs[k].missed && jobs[k].rem > 0 && jobs[k].dl <= simLen) {
            jobs[k].missed = 1; jobs[k].remAtMiss = jobs[k].rem; missOrder[nm++] = k;
        }

    printf("\n--- Simulation over hyperperiod H = %ld (simulated %d units) ---\n", H, simLen);
    printf("Timeline (start-job-end):\n");
    int col = 0;
    for (int u = 0; u < simLen; u++)
        if (brk[u]) {
            printf("%d-%s-", u, lab[u]);
            if (++col % 8 == 0) printf("\n");
        }
    printf("%d\n", simLen);

    printf("\nMax observed response time per task (simulation vs RTA):\n");
    for (int i = 0; i < n; i++) {
        int mx = 0;
        for (int k = 0; k < nj; k++)
            if (jobs[k].task == i && jobs[k].fin > 0 && jobs[k].fin - jobs[k].rel > mx)
                mx = jobs[k].fin - jobs[k].rel;
        printf("  %s: observed %d, RTA %d%s\n", id[i], mx, R[i],
               R[i] > Tp[i] ? " (RTA stopped once R exceeded the deadline)" : "");
    }

    printf("\nMissed deadlines:\n");
    if (nm == 0) printf("  None\n");
    for (int m = 0; m < nm; m++) {
        Job *j = &jobs[missOrder[m]];
        printf("  %s job %d (rel. %d) missed deadline t=%d, %d unit(s) left, ",
               id[j->task], j->num, j->rel, j->dl, j->remAtMiss);
        if (j->fin > 0) printf("finished late at t=%d\n", j->fin);
        else printf("unfinished at end\n");
```

```
    }
    return 0;
}
```

OUTPUT:

For case-1 (sample input 1, schedulable): T1(3,8), T2(2,10), T3(2,14)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in4_a.txt
3
T1 3 8 0
T2 2 10 0
T3 2 14 0
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rms < in4_a.txt
Task   C    T  Phase (listed in priority order)
T1     3    8      0
T2     2   10      0
T3     2   14      0

--- Schedulability analysis ---
Utilization U = 3/8 + 2/10 + 2/14 = 0.7179
Liu-Layland bound n(2^(1/n)-1) = 0.7798
Utilization test: U <= bound, schedulability GUARANTEED

Response-time analysis:
  T1: R = 3 -> 3 => R = 3, deadline = 8 : schedulable
  T2: R = 2 -> 5 -> 5 => R = 5, deadline = 10 : schedulable
  T3: R = 2 -> 7 -> 7 => R = 7, deadline = 14 : schedulable
RTA conclusion: task set is SCHEDULABLE

--- Simulation over hyperperiod H = 280 (simulated 280 units) ---
Timeline (start-job-end):
0-T1.1-3-T2.1-5-T3.1-7-IDLE-8-T1.2-11-T2.2-13-IDLE-14-T3.2-
16-T1.3-19-IDLE-20-T2.3-22-IDLE-24-T1.4-27-IDLE-28-T3.3-30-T2.4-
32-T1.5-35-IDLE-40-T1.6-43-T2.5-45-T3.4-47-IDLE-48-T1.7-51-T2.6-
53-IDLE-56-T1.8-59-T3.5-60-T2.7-62-T3.5-63-IDLE-64-T1.9-67-IDLE-
70-T2.8-72-T1.10-75-T3.6-77-IDLE-80-T1.11-83-T2.9-85-T3.7-87-IDLE-
88-T1.12-91-T2.10-93-IDLE-96-T1.13-99-T3.8-100-T2.11-102-T3.8-103-IDLE-
104-T1.14-107-IDLE-110-T2.12-112-T1.15-115-T3.9-117-IDLE-120-T1.16-123-T2.13-
125-IDLE-126-T3.10-128-T1.17-131-T2.14-133-IDLE-136-T1.18-139-IDLE-140-T2.15-
142-T3.11-144-T1.19-147-IDLE-150-T2.16-152-T1.20-155-T3.12-157-IDLE-160-T1.21-
163-T2.17-165-IDLE-168-T1.22-171-T2.18-173-T3.13-175-IDLE-176-T1.23-179-IDLE-
180-T2.19-182-T3.14-184-T1.24-187-IDLE-190-T2.20-192-T1.25-195-IDLE-196-T3.15-
198-IDLE-200-T1.26-203-T2.21-205-IDLE-208-T1.27-211-T2.22-213-T3.16-215-IDLE-
216-T1.28-219-IDLE-220-T2.23-222-IDLE-224-T1.29-227-T3.17-229-IDLE-230-T2.24-
232-T1.30-235-IDLE-238-T3.18-240-T1.31-243-T2.25-245-IDLE-248-T1.32-251-T2.26-
253-T3.19-255-IDLE-256-T1.33-259-IDLE-260-T2.27-262-IDLE-264-T1.34-267-T3.20-
269-IDLE-270-T2.28-272-T1.35-275-IDLE-280

Max observed response time per task (simulation vs RTA):
  T1: observed 3, RTA 3
  T2: observed 5, RTA 5
  T3: observed 7, RTA 7

Missed deadlines:
  None
```

For case-2 (sample input 2, unschedulable): T1(4,5), T2(3,7)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in4_b.txt
```

```
2
T1 4 5 0
T2 3 7 0
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rms < in4_b.txt
Task   C    T  Phase (listed in priority order)
T1     4    5      0
T2     3    7      0

--- Schedulability analysis ---
Utilization U = 4/5 + 3/7 = 1.2286
Liu-Layland bound n(2^(1/n)-1) = 0.8284
Utilization test: U > 1, definitely UNSCHEDULABLE

Response-time analysis:
  T1: R = 4 -> 4 => R = 4, deadline = 5 : schedulable
  T2: R = 3 -> 7 -> 11 => R = 11, deadline = 7 : MISSES DEADLINE
RTA conclusion: task set is NOT schedulable

--- Simulation over hyperperiod H = 35 (simulated 35 units) ---
Timeline (start-job-end):
0-T1.1-4-T2.1-5-T1.2-9-T2.1-10-T1.3-14-T2.1-15-T1.4-19-T2.2-
20-T1.5-24-T2.2-25-T1.6-29-T2.2-30-T1.7-34-T2.3-35

Max observed response time per task (simulation vs RTA):
  T1: observed 4, RTA 4
  T2: observed 23, RTA 11 (RTA stopped once R exceeded the deadline)

Missed deadlines:
  T2 job 1 (rel. 0) missed deadline t=7, 2 unit(s) left, finished late at t=15
  T2 job 2 (rel. 7) missed deadline t=14, 3 unit(s) left, finished late at t=30
  T2 job 3 (rel. 14) missed deadline t=21, 3 unit(s) left, unfinished at end
  T2 job 4 (rel. 21) missed deadline t=28, 3 unit(s) left, unfinished at end
  T2 job 5 (rel. 28) missed deadline t=35, 3 unit(s) left, unfinished at end
```

For case-3: utilization above the bound but still schedulable: T1(1,4), T2(2,6), T3(3,12)

```
aditya-forge@LAPTOP-184D82EB:~/os_lab$ cat in4_c.txt
3
T1 1 4 0
T2 2 6 0
T3 3 12 0
aditya-forge@LAPTOP-184D82EB:~/os_lab$ ./rms < in4_c.txt
Task   C    T  Phase (listed in priority order)
T1     1    4      0
T2     2    6      0
T3     3   12      0

--- Schedulability analysis ---
Utilization U = 1/4 + 2/6 + 3/12 = 0.8333
Liu-Layland bound n(2^(1/n)-1) = 0.7798
Utilization test: bound < U <= 1, NOT GUARANTEED (inconclusive, check RTA)

Response-time analysis:
  T1: R = 1 -> 1 => R = 1, deadline = 4 : schedulable
  T2: R = 2 -> 3 -> 3 => R = 3, deadline = 6 : schedulable
  T3: R = 3 -> 6 -> 7 -> 9 -> 10 -> 10 => R = 10, deadline = 12 : schedulable
RTA conclusion: task set is SCHEDULABLE

--- Simulation over hyperperiod H = 12 (simulated 12 units) ---
Timeline (start-job-end):
```

```
0-T1.1-1-T2.1-3-T3.1-4-T1.2-5-T3.1-6-T2.2-8-T1.3-9-T3.1-
10-IDLE-12

Max observed response time per task (simulation vs RTA):
  T1: observed 1, RTA 1
  T2: observed 3, RTA 3
  T3: observed 10, RTA 10

Missed deadlines:
  None
```

REPORT:

- Method: tasks get priority by period, shortest first. The utilization test says guaranteed if U ≤ bound, definitely unschedulable if U > 1 and not guaranteed in between. In that middle case the response-time analysis decides, and the set is schedulable only if every Ri ≤ Ti. The simulation moves one time unit at a time, runs the highest-priority ready job and records any job still unfinished at its deadline. A late job keeps running; it is not dropped.
- Case-1: U = 0.7179 is below the bound 0.7798, so the set is guaranteed schedulable. The response times are R1 = 3, R2 = 5 and R3 = 7, all within their periods. The simulation over H = 280 shows no missed deadline, and the worst response times seen in the simulation equal the RTA values.
- Case-2: U = 1.2286 is above 1, so the set is definitely unschedulable. In the RTA, T2 goes 3 → 7 → 11, which is above its period of 7. In the simulation T2 misses its deadline at t = 7, 14, 21, 28 and 35, and its backlog grows because T1 uses 80% of the CPU.
- Case-3: U = 0.8333 is above the bound 0.7798, so the utilization test is inconclusive. The response-time analysis gives R3 = 10 ≤ 12, so the set is schedulable, and the simulation confirms it with no missed deadline. This shows that the Liu-Layland bound is sufficient but not necessary.